МИНСПОРТ РОССИИ

ФСП федерация спортивного программирования России

КУБОК ФЕДЕРАЦИИ 2026

РАЙТАП · REVERSE

LICENSE

REVERSE

19 СЕНТЯБРЯ 2026
ОНЛАЙН · CTF TASK-BASED

REVERSE
КАТЕГОРИЯ

14
СЛАЙДОВ В РАЗБОРЕ

01

РАЗВЕДКА

Клиент раздаётся только с сайта — качаем `MosMetroAutoPilot-2.0-win64.exe`, запускаем.

Картина знакомая: «клиент с лицензией». На странице загрузки честно лежит демо-ключ `MMAP-DEMO-2026-7DAY`. Вводим — приложение отвечает **«Demo-доступ активен»**, открывает просмотр и метрики, но управление составом и вкладка Pro закрыты, а флага нет. Значит флаг — за режимом **PRO**, и вся задача сводится к одному вопросу: при каком условии клиент решает, что у нас Pro.

Ключевая деталь, которая сразу задаёт жанр: это **не крэкми с проверкой пароля внутри**. Клиент ничего не считает у себя — он **спрашивает сервер лицензий** и верит ответу. А раз есть тот, кому доверяют, — есть и канал, который можно перехватить. Зайдём с двух сторон: сначала сайт, потом сам бинарь.

LICENSE
REVERSE

02

ЧТО ГОВОРИТ САЙТ (И ЧЕГО НЕ ГОВОРИТ)

Открываем DevTools на сайте, вкладка Network — при загрузке страницы уходит запрос к сервису лицензий. Он же виден в `assets/js/status.js`:

```
const ACTIVATION_BASE = "https://license.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai";
fetch(`${ACTIVATION_BASE}/status`, { headers: { Accept: "application/json" } });
```

Сайт выдаёт ровно одну полезную вещь — **хост активации** `license.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai`. Токена тут нет намеренно: светить его в клиентском JS бессмысленно, и, как мы увидим ниже, его значение атакующему вообще не нужно.

Это Punycode-запись домена `license.quals.кубокфедерации2026.рф`. В `hosts` и командах ниже используется ASCII-вариант.

(Хост можно узнать и без сайта: он лежит открытой строкой в бинаре (`Provisioning.Host`), а в Wireshark виден по DNS/SNI в TLS ClientHello. Сайт лишь экономит время на recon — сам протокол мы всё равно читаем из клиента.)

LICENSE
REVERSE

03

КАК КЛИЕНТ ПРОВЕРЯЕТ ЛИЦЕНЗИЮ (ВЗГЛЯД В ДЕКОМПАЙЛ)

Бинарь — .NET (Avalonia), так что берём dnSpy/ILSpy и читаем `Activation.cs`. Логика короткая и на удивление доверчивая. Сначала — как создаётся HTTP-клиент:

```
var handler = new SocketsHttpHandler {
    UseProxy = false,
    SslOptions = new SslClientAuthenticationOptions {
        // принимаем ЛЮБОЙ сертификат сервера
        RemoteCertificateValidationCallback = static (_, _, _, _) ⇒ true
    }
};
```

Вот и первая половина уязвимости открытым текстом: клиент **отключил проверку TLS- сертификата** — верёвка, на которой всё и повесится. Дальше — сам запрос и вердикт:

LICENSE
REVERSE

03

КАК КЛИЕНТ ПРОВЕРЯЕТ ЛИЦЕНЗИЮ (ВЗГЛЯД В ДЕКОМПАЙЛ)

ПРОДОЛЖЕНИЕ

```
request.Headers.Add("X-Admin-Token", expectedToken);          // токен зашит в самом клиенте (обфусцирован)
using var response = await Http.SendAsync(request, ...);       // POST https://license.quals.xn--2026-43dfv
hb5aarb9c8a4bu5b.xn--p1ai/check
if (response.StatusCode ≠ HttpStatusCode.OK) return Error(ServerError);

if (!HasExpectedToken(response, expectedToken))                // сервер ДОЛЖЕН вернуть токен обратно
    return Error(AuthenticationError);

if (!TryReadSuccessfulResponse(json, out client, out expiresAt, out plan))
    return Error(InvalidLicenseError);

var flag = string.Equals(plan, "pro", StringComparison.OrdinalIgnoreCase)
    ? Provisioning.Unseal()                                    // ← вот здесь рождается флаг
    : null;
```

Разбираем гейт по косточкам — что именно должен вернуть сервер, чтобы клиент открыл Pro:

LICENSE
REVERSE

03

КАК КЛИЕНТ ПРОВЕРЯЕТ ЛИЦЕНЗИЮ (ВЗГЛЯД В ДЕКОМПАЙЛ)

ПРОДОЛЖЕНИЕ

- **200 OK** — иначе `ServerError`.
- Заголовок ответа `X-Admin-Token`, равный ожидаемому (`HasExpectedToken`) — «проверка подлинности сервера». Тонкость: клиент сверяет эхо со **своим встроенным** токеном, а его же и шлёт в запросе — значит фейковому серверу достаточно **отразить заголовок из запроса**, и знать значение токена не нужно вовсе. Это **CWE-287 Improper Authentication**: «доказательством» подлинности служит то, что клиент прислал сам.
- JSON, который `TryReadSuccessfulResponse` признаёт валидным: `valid`, `active`, `exists` — булев `true`; `days_left` — **положительное целое**; `client`, `expires_at`, `plan` — **непустые строки**. Типы строгие: строка вместо `true` или `"7"` вместо `7` — уже отказ.
- И, наконец, единственная развилка на весь флаг: `plan == "pro"` → зовётся `Provisioning.Unseal()`.

Резюме гейта, без лишних слов: **вернуть 200, эхом отдать публичный токен и прислать `plan:"pro"` в правильно типизированном JSON — и клиент сам достанет флаг.** Ни привязки к «настоящей» лицензии, ни серверной подписи — чистое доверие к тексту, пришедшему по каналу, который к тому же не аутентифицирован.

(Отключённый `RemoteCertificateValidationCallback ⇒ true` — это **CWE-295 Improper Certificate Validation**, живучая классика: так не раз прилетало реальным телеметрия- и лицензия-клиентам, которые «временно» глушили валидацию сертификата и уезжали с этим в релиз. TLS без проверки сертификата защищает ровно ни от чего — шифрование есть, а вот «с тем ли ты сервером» — уже нет.)

03

КАК КЛИЕНТ ПРОВЕРЯЕТ ЛИЦЕНЗИЮ (ВЗГЛЯД В ДЕКОМПАЙЛ)

ПРОДОЛЖЕНИЕ

LICENSE
REVERSE

(Заодно ответ на популярный вопрос «а не увидеть ли токен и ответ в Wireshark?». Хост — да: DNS-запрос и **SNI** в TLS ClientHello идут открыто, это честный recon. А вот тело `/check`, заголовок и JSON — под TLS. Вытащить сессионные ключи тоже не выйдет: .NET на Windows ходит через SChannel и не пишет `SSLKEYLOGFILE`, в отличие от браузеров. Так что пассивный дамп покажет «клиент стучится на license.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai:443» — и всё. Чтобы прочитать разговор, надо не подслушивать, а **стать сервером**.)

04

САМО-MITM: ПОДМЕНЯЕМ СЕРВЕР ЛИЦЕНЗИЙ

Раз клиент безоговорочно верит серверу (0x03), а легально Pro никто не выдаёт — станем этим сервером сами. Эксплойт собирается из трёх частей, все снаружи бинаря, ничего в файле не патчим.

Часть 1. Перенаправляем хост на себя. Клиент ходит на `https://license.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai` — заставим это имя резолвиться в нашу машину. В `C:\Windows\System32\drivers\etc\hosts`:

```
127.0.0.1   license.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai
```

Часть 2. Любой сертификат. Клиент принимает какой угодно серт (0x03), так что ставить его в доверенные **не нужно** — хватит самоподписанного:

```
python solver/gen_cert.py license.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai     # cert.pem + key.pem
```

Часть 3. Фейковый сервер лицензий, который отвечает на `/check` ровно тем, что требует гейт из 0x03. Здесь важно не срезаться на двух деталях — именно на них чаще всего и спотыкаются:

LICENSE
REVERSE

04

САМО-MITM: ПОДМЕНЯЕМ СЕРВЕР ЛИЦЕНЗИЙ

ПРОДОЛЖЕНИЕ

```
# суть ответа (полностью — в solver/solve_server.py)
self.send_response(200)
self.send_header("X-Admin-Token", self.headers.get("X-Admin-Token"))  # ← ДЕТАЛЬ 1: отражаем токен из запро
са
self.wfile.write(json.dumps({
    "valid": True, "active": True, "exists": True,
    "expires_at": "2099-12-31T23:59:59Z",
    "days_left": 3650, "client": "CTF",
    "plan": "pro"                                          # ← ДЕТАЛЬ 2: именно "pro"
}).encode())
```

- **Деталь 1.** Токен нужно **вернуть в заголовке ОТВЕТА** — проще всего отразить тот, что пришёл в запросе (значение знать не нужно). Забыл эхо → `HasExpectedToken` = false → `Сбой аутентификации сервера лицензий`. (Классическая ловушка: принимают токен в запросе, но не возвращают в ответе — и недоумевают, почему «сервер не аутентичен».)
- **Деталь 2.** Без поля `plan` (или с чем угодно, кроме `"pro"`) `TryReadSuccessfulResponse` вернёт отказ → `Ключ недействителен`, а если и пройдёт — флага не будет. Нужен именно `"plan":"pro"`.

Ключ при активации можно вводить **любой** — клиент не проверяет его сам, решение о Pro целиком за сервером, а сервер теперь наш.

LICENSE
REVERSE

04

САМО-MITM: ПОДМЕНЯЕМ СЕРВЕР ЛИЦЕНЗИЙ

ПРОДОЛЖЕНИЕ

(С точки зрения каталога слабостей это связка **CWE-295** (приняли любой серт) и доверия к неаутентифицированному источнику: имея перенаправление имени и «любой сертификат сойдёт», мы делаем **MITM против самих себя** — становимся сервером, которому клиент верит без оговорок. Настоящий TLS тут не помеха: мы не взламываем канал, мы им легально владеем.)

LICENSE
REVERSE

05

ПОЧЕМУ ФЕЙКОВЫЙ СЕРВЕР ОТДАЁТ НАСТОЯЩИЙ ФЛАГ

Читаем `Provisioning.Unseal()` — и видим, что **сервер на ключ вообще не влияет**. Флаг лежит в бинаре, а весь ключевой материал выводится из данных **внутри файла**:

```
// токен восстанавливается из бинаря (обфускация — XOR по ключу "MMAP")
var token = Encoding.ASCII.GetString( Tk[i] ^ Mk[i % Mk.Length] );   // = 7a8f9c3e...

var h   = SHA256(token);
var key = SHA256(token ‖ h);
for (i = 0; i < 6007; i++) key = SHA256(key ‖ h);                    // самопальная растяжка ключа

// поток-маска: SHA256(key ‖ nonce ‖ counter_be32) ...
var cipher = Bl[i] ^ keystream[i];                                   // хранится НЕ чистый AES-шифртекст
var flag   = AES_CBC_decrypt(key, Iv, cipher);                       // → flag{...}
```

Сервер в этой цепочке не участвует нигде. Его единственная работа — **довести исполнение до ветки** `plan:"pro"`, где ключ давно предопределён самим файлом. Поэтому оба пути — хоть наш само-MITM, хоть честная статическая распаковка `Unseal` руками — выдают **буквально один и тот же токен**. Подмена сервера не подделывает флаг, она лишь открывает дверь в комнату, где он всегда лежал.

LICENSE
REVERSE

05

ПОЧЕМУ ФЕЙКОВЫЙ СЕРВЕР ОТДАЁТ НАСТОЯЩИЙ ФЛАГ

ПРОДОЛЖЕНИЕ

(Тут стоит быть честным про «защиту». Клиентский флаг — это всегда вопрос времени, а не стойкости: раз клиент умеет его показать, реверсер умеет его достать. Кастомный KDF — растяжка `SHA256` на 6007 раундов, соль, выведенная из токена, и маска-keystream поверх AES — это **поднятие планки, а не секрет**: убрана тривиальная дорожка «нашёл `flag{` /чистый AES-блоб → расшифровал в одну строку». По духу это самопальный родственник PBKDF2 — не для секретности, а чтобы детерминированно перемешать известные файлу байты; см. `solver/unseal_reference.py`, он проделывает ровно это и достаёт флаг офлайн.)

LICENSE
REVERSE

06

ЗАПУСК И ФЛАГ

Собираем всё вместе и активируемся любым ключом:

```
# 1) hosts:  127.0.0.1  license.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai
# 2) cert:
python solver/gen_cert.py license.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--p1ai
# 3) сервер (нужен :443):
python solver/solve_server.py
```

Pro включился не потому, что у нас есть лицензия, а потому, что «сервер лицензий» теперь диктуем мы — и на любой ключ он говорит `pro`. Автопилот в легенде научился водить состав сам; мы, по-своему, научили клиент активироваться сам.

`flag{2026_tr41n_l34rns_t0_dr1v3_1ts3lf}`

> **Категория:** Reverse · **Сложность:** medium · **Время:** 30–60 мин Флаг хранится только в `../flag.txt`. Референс-эксплойт — `solver/solve_server.py` (+ `solve_server.ps1`), офлайн-декрипт — `solver/unseal_reference.py`.

LICENSE
REVERSE

ФЛАГ

```
flag{2026_tr41n_l34rns_t0_dr1v3_1ts3lf}
```

Задание «License» решено